```
    deliver(email: user_email, subject: subject, body: body)
  end

  def deliver(email:, subject:, body:)
    # send the message via SMTP
  end
end
```

Here's an RSpec example for this class that checks the content of the email. We don't want to send real email from our specs, so on the highlighted lines we're mocking out the deliver method. Our version will check that we're calling it with the right body but not actually send an email:

15-using-test-doubles-effectively/03/daily_summary_email/spec/daily_summary_spec.rb

```
RSpec.describe DailySummary do
  let(:todays_messages) do
    [
      { thread_id: 1, content: 'Hello world' },
      { thread_id: 2, content: 'I think forums are great' },
      { thread_id: 2, content: 'Me too!' }
    ]
  end

  it "sends a summary of today's messages and threads" do
    summary = DailySummary.new

➤   expect(summary).to receive(:deliver).with(
➤     hash_including(body: 'You missed 3 messages in 2 threads today')
➤   )

    summary.send_daily_summary('user@example.com', todays_messages)
  end
end
```

Alas, this spec exhibits the stubject code smell. We're faking out the deliver method but testing the real send_daily_summary method on the same object.

Test doubles are intended to help you construct a test environment for a real object. Specs like this one blur the line between the subject and its environment. The temptation to fake out part of an object with allow or expect is a design signal. In other words, it's a hint that one object has two responsibilities, and might result in a better design if we split it up.

First, we can put the SMTP logic into its own EmailSender class:

15-using-test-doubles-effectively/04/daily_summary_email/lib/email_sender.rb

```
class EmailSender
  def deliver(email:, subject:, body:)
    # send the message via SMTP
  end
end
```